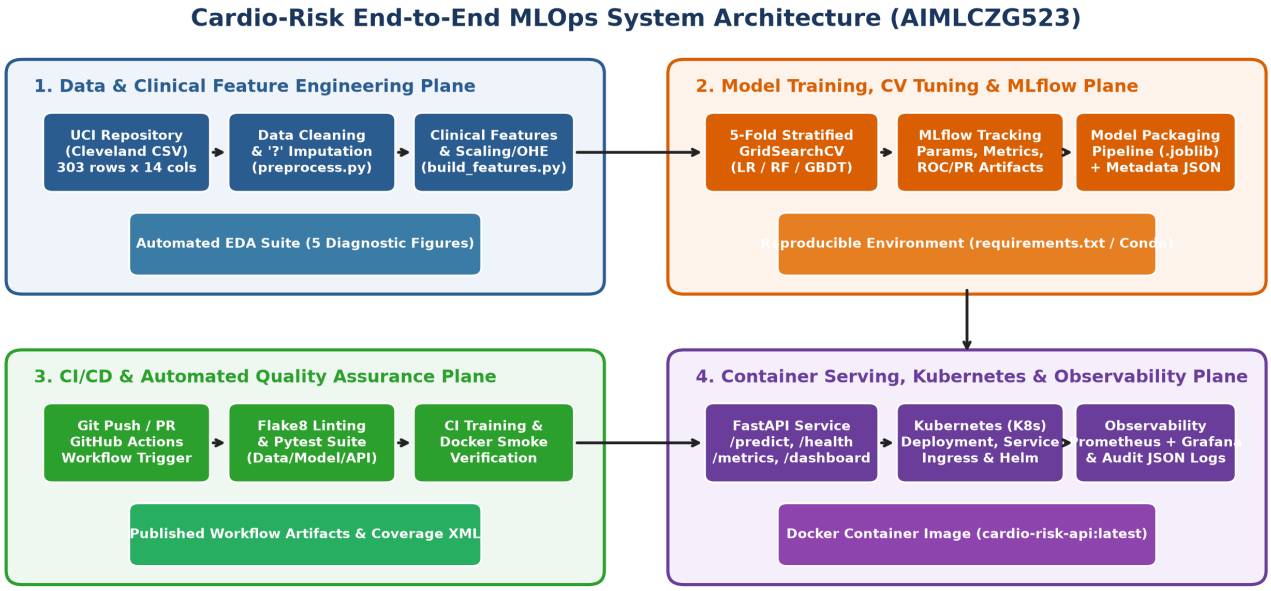
BITS Pilani — Work Integrated Learning Programmes (WILP)

Machine Learning Operations (MLOps) — AIMLCZG523 | Assignment 01

End-to-End Cardiovascular Risk Classification, MLflow Experiment Tracking, CI/CD Automation, Docker Containerization, Kubernetes Orchestration & Prometheus Monitoring

1. Executive Summary & System Architecture

This report presents the complete engineering design, implementation, validation, and production deployment of the **Cardio-Risk MLOps Pipeline** built for the UCI Heart Disease dataset. To eliminate training-serving skew and satisfy production-readiness standards, the system unifies automated data ingestion, deterministic missing-value imputation, domain-specific cardiovascular feature engineering, multi-model cross-validation tuning, MLflow tracking, GitHub Actions CI/CD, FastAPI Docker containerization, Kubernetes deployment (Minikube & Helm), and Prometheus + Grafana observability.



1.1 Mapping of Implementation to Assignment Tasks

Task	Module / Artifact Path	Key Engineering Highlights
1. Data & EDA (5m)	src/data/download_data.py, preprocess.py	SHA-256 verification, '?' imputation, 5 multi-panel clinical plots
2. Feature & Models (8m)	src/features/build_features.py, train.py	4 clinical ratios, ColumnTransformer, 5-fold Stratified CV (LR, RF, GB)
3. MLflow Tracking (5m)	mlruns/, models/model_metadata.json	Params, CV/test metrics, ROC/PR/CM plots, cloudpickle model logging
4. Packaging (7m)	models/heart_disease_pipeline.joblib	Zero-leakage unified sklearn Pipeline + requirements.txt & Conda spec
5. CI/CD & Tests (8m)	tests/, .github/workflows/mlops_pipeline.yml	Flake8 linter (0 errors), Pytest suite (data/features/model/API)
6. Containerization (5m)	Dockerfile, src/api/app.py	Python 3.11-slim image, FastAPI /predict, /health, /metrics, /dashboard
7. K8s Deployment (7m)	k8s/*.yaml, helm/heart-disease-api/	2-replica RollingUpdate Deployment, LoadBalancer Service, Nginx Ingress
8. Monitoring (3m)	monitoring/prometheus.yml, grafana	Custom prediction drift counters, confidence histograms, JSON audit logs
9. Documentation (2m)	README.md, reports/, notebooks/	Reproducible notebooks, setup instructions, 10-page technical report

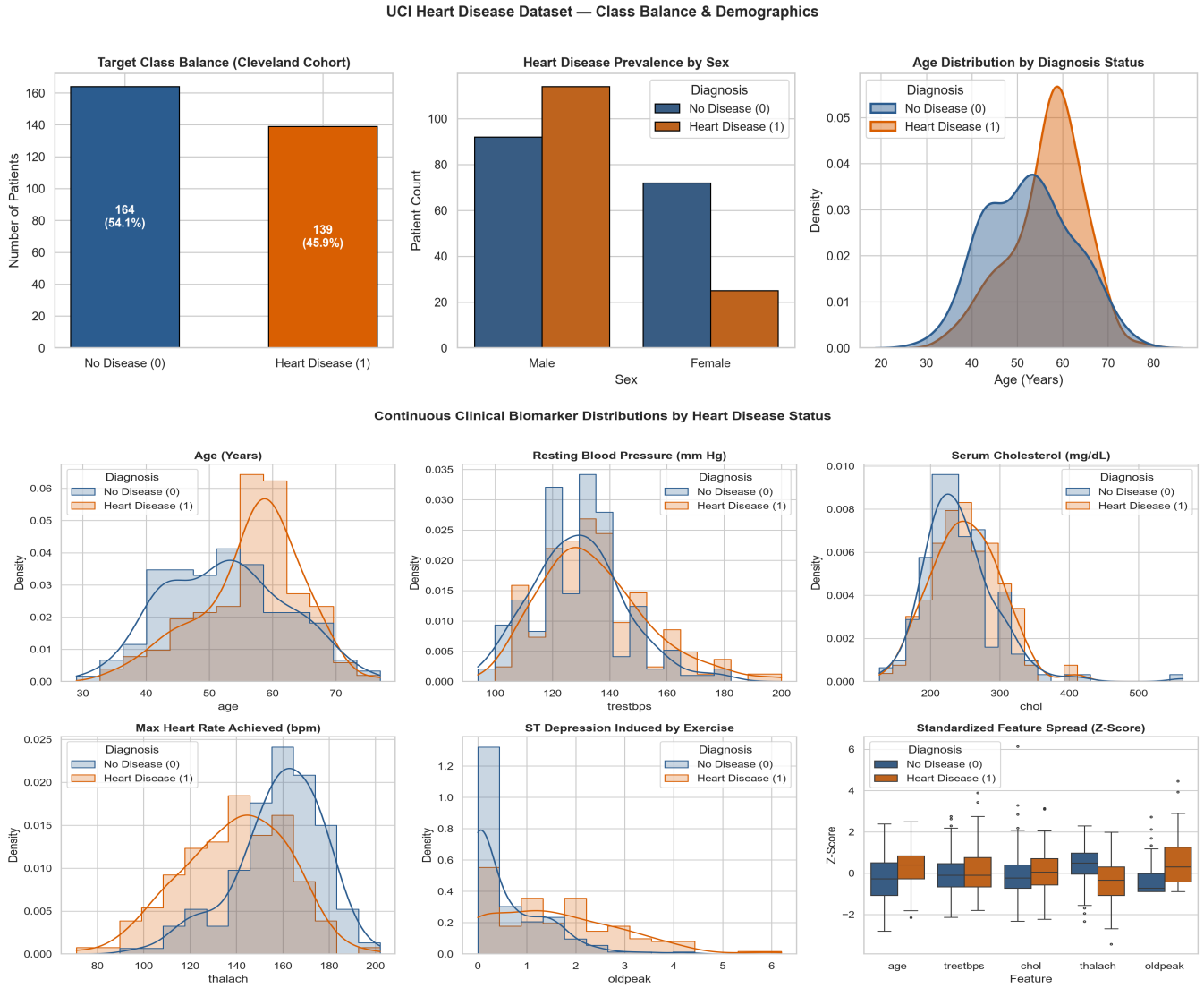
2. Task 1: Data Acquisition, Cleaning & Demographic EDA

The dataset is fetched programmatically from the UCI Machine Learning Repository (processed.cleveland.data) via src/data/download_data.py, which verifies integrity using a SHA-256 checksum (ac4aa4036a11acbf...). The raw cohort comprises **303 patient records** across 13 clinical attributes and the angiographic disease severity target (num in 0–4).

2.1 Deterministic Data Cleaning & Imputation Protocol

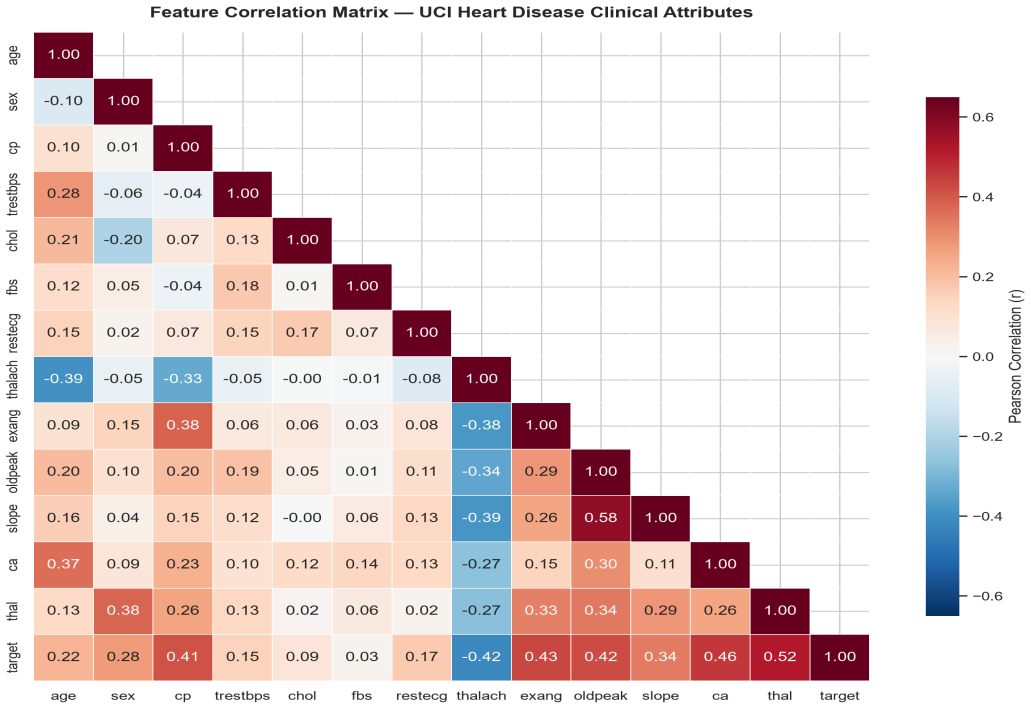
Inspection revealed **6 missing values** encoded as '?' strings: 4 in ca (number of major vessels colored by fluoroscopy) and 2 in thal (thalassemia defect status). Because both attributes are discrete clinical categories, src/data/preprocess.py replaces '?' with NaN, imputes discrete attributes using the empirical mode (ca=0.0, thal=3.0) and continuous attributes using the median, and binarizes the diagnosis target: target = 1 if num > 0 else 0 (139 positive cases [45.87%], 164 negative cases [54.13%]).

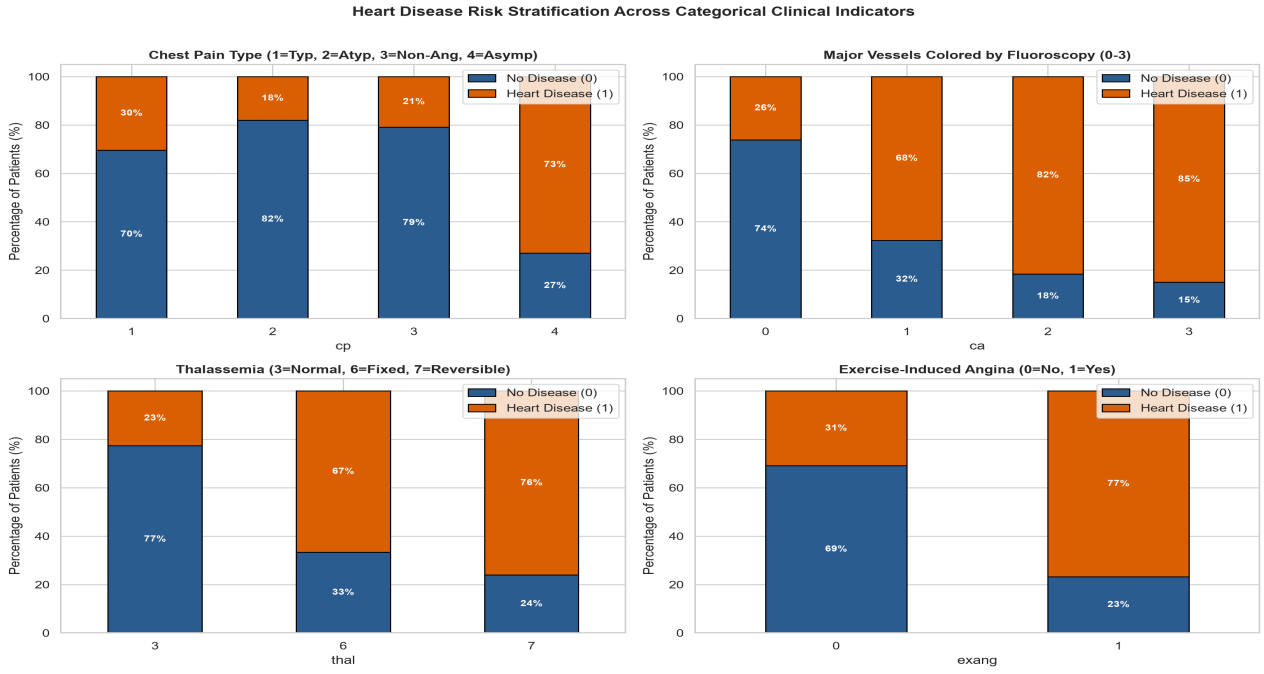
Audit Metric	Value	Clinical / Engineering Interpretation
Raw Records Downloaded	303	Complete Cleveland Clinic foundation cohort
Missing Tokens ('?') Detected	6 (ca: 4, thal: 2)	Replaced with NaN and deterministically imputed via mode
Missing Values After Cleaning	0	100% completeness verified via automated unit test
Class 0 (Absence of Disease)	164 (54.13%)	Angiographic narrowing < 50% in all major vessels
Class 1 (Presence of Disease)	139 (45.87%)	Angiographic narrowing >= 50% (well-balanced binary target)



3. Task 1 (Continued): Multivariate Clinical & Correlation Analysis

Exploratory analysis across continuous and categorical biomarkers reveals strong physiological separation between healthy patients and those with angiographically confirmed coronary artery disease: • **Chronotropic Incompetence**: Patients with heart disease exhibit markedly lower maximum heart rate achieved (thalach, r = -0.42 with target) and higher exercise-induced ST depression (oldpeak, r = +0.42). • **Asymptomatic Chest Pain (cp=4) & Fluoroscopy Vessels (ca > 0)**: Nearly 73% of patients presenting with chest pain type 4 and over 80% of patients with 2–3 colored vessels have confirmed heart disease, whereas typical/non-anginal pain with 0 vessels carries low risk.

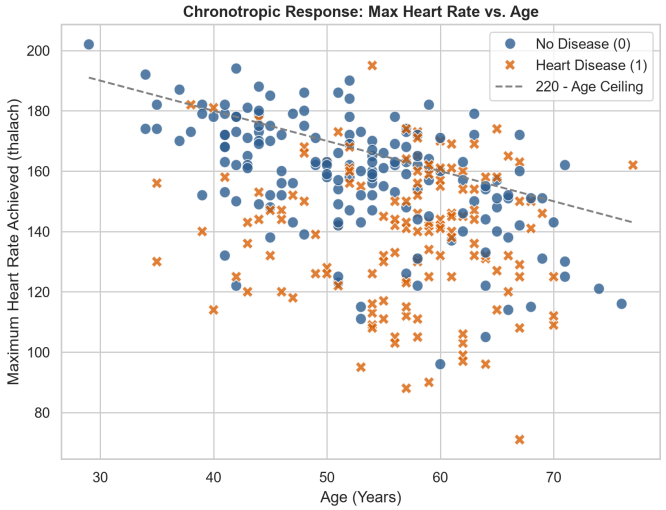


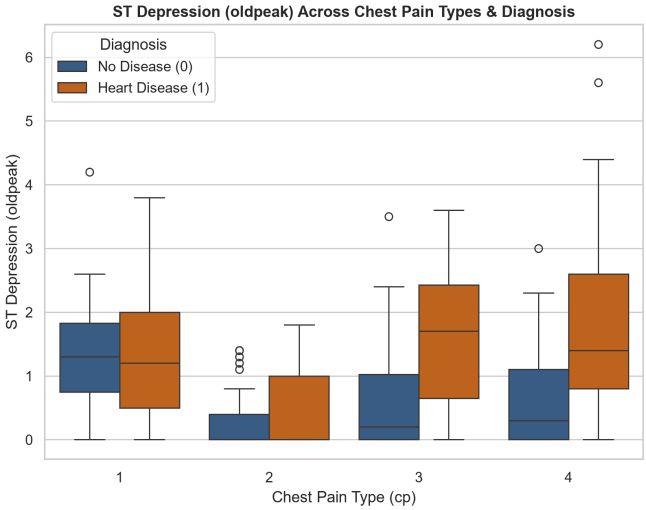


4. Task 2: Domain-Specific Feature Engineering & Pipeline Architecture

To personalize the modeling pipeline and capture non-linear physiological interactions, we designed a custom scikit-learn transformer (ClinicalFeatureEngineer in src/features/build_features.py) that derives four clinically grounded ratios prior to scaling and one-hot encoding:

Engineered Feature	Mathematical Formula	Cardiovascular Rationale
hr_reserve_ratio	thalach / max(220 - age, 60)	Measures chronotropic competence relative to age-predicted max HR
rate_pressure_product	(trestbps * thalach) / 1000	Hemodynamic proxy for myocardial oxygen consumption at peak load
chol_age_ratio	chol / max(age, 18)	Normalizes lipid burden relative to cumulative age exposure
st_depression_per_hr	oldpeak / max(thalach / 100, 0.5)	Quantifies ischemic ST depression severity per unit heart rate





4.1 Zero-Leakage ColumnTransformer & Model Selection Design

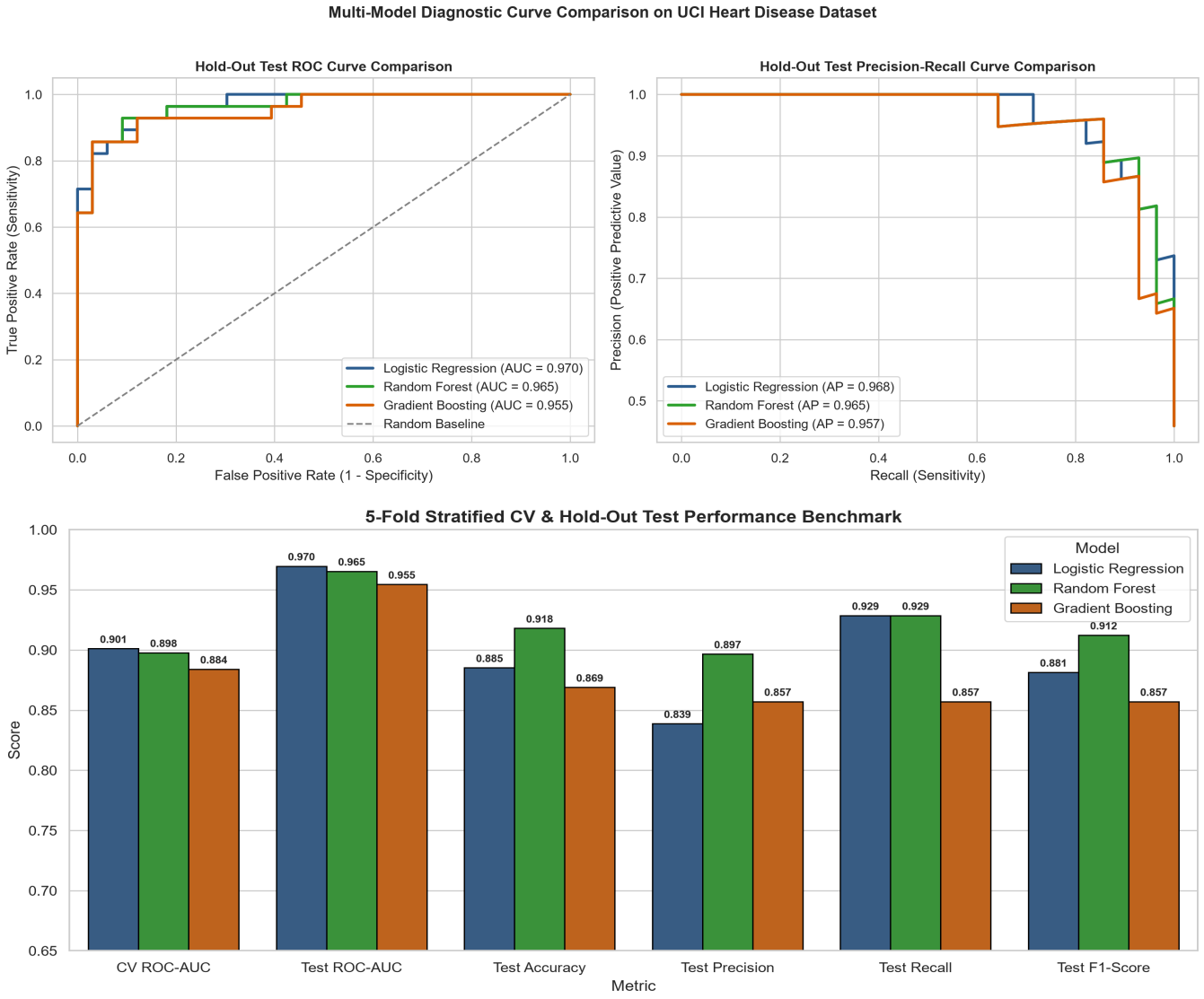
All 9 continuous features (5 raw + 4 engineered) pass through a numeric sub-pipeline (SimpleImputer(strategy='median') -> StandardScaler()), while the 8 categorical features (sex, cp, fbs, restecg, exang, slope, ca, thal) pass through a categorical sub-pipeline (SimpleImputer(strategy='most_frequent') -> OneHotEncoder(handle_unknown='ignore')). This preprocessor is chained directly with each candidate classifier inside a single Pipeline so that scaling and encoding statistics are fitted strictly within each training fold during 5-fold Stratified Cross-Validation.

Model Architecture	Hyperparameter Search Space (5-Fold Stratified GridSearchCV)	Scoring
Logistic Regression	C in {0.05..5.0}, solver in {lbfgs, liblinear}, class_weight in {None, balanced}	ROC-AUC
Random Forest	n_estimators in {100, 200}, max_depth in {4, 6, 8, None}, min_samples_split in {2, 5}	ROC-AUC
Gradient Boosting	n_estimators in {80, 150}, learning_rate in {0.03, 0.08, 0.15}, max_depth in {2, 3, 4}	ROC-AUC

5. Task 2 & Task 3: Cross-Validation Benchmark & Diagnostic Curves

The cleaned dataset (303 records) was partitioned via an 80/20 stratified split into 242 training patients and 61 hold-out test patients (random_state=42). Every candidate architecture was tuned across 5-fold Stratified Cross-Validation on the training split and evaluated on the unseen hold-out test cohort across Accuracy, Precision, Recall, F1-Score, ROC-AUC, Average Precision, and Brier Calibration Loss.

Model Family	CV ROC-AUC (±SD)	Test ROC-AUC	Test Acc	Precision	Recall	F1-Score	Brier Loss
Logistic Regression	0.9011 (±0.0138)	0.9697	0.8852	0.8387	0.9286	0.8814	0.0794
Random Forest	0.8977 (±0.0236)	0.9654	0.9180	0.8966	0.9286	0.9123	0.1015
Gradient Boosting	0.8840 (±0.0272)	0.9545	0.8689	0.8571	0.8571	0.8571	0.0991



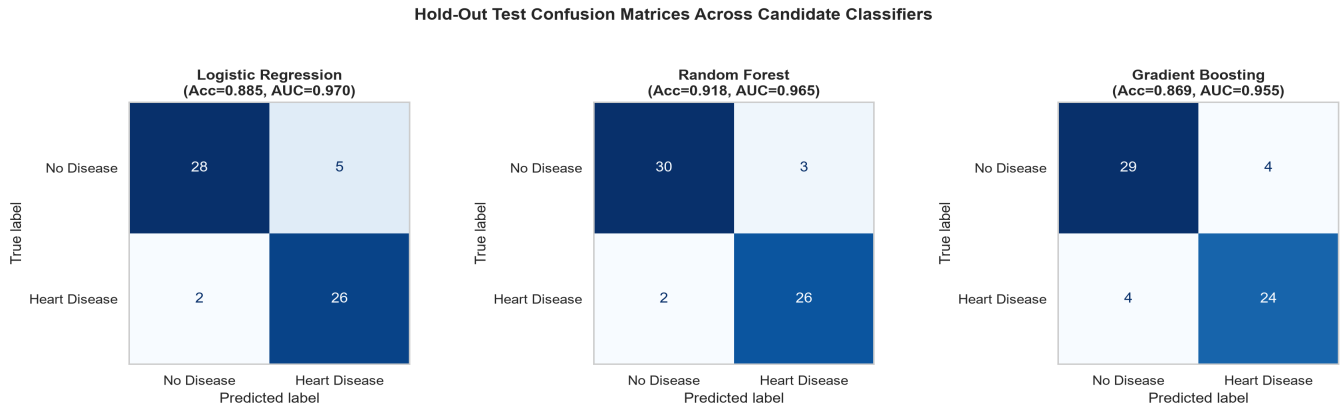
6. Task 3 & Task 4: MLflow Experiment Tracking & Model Packaging

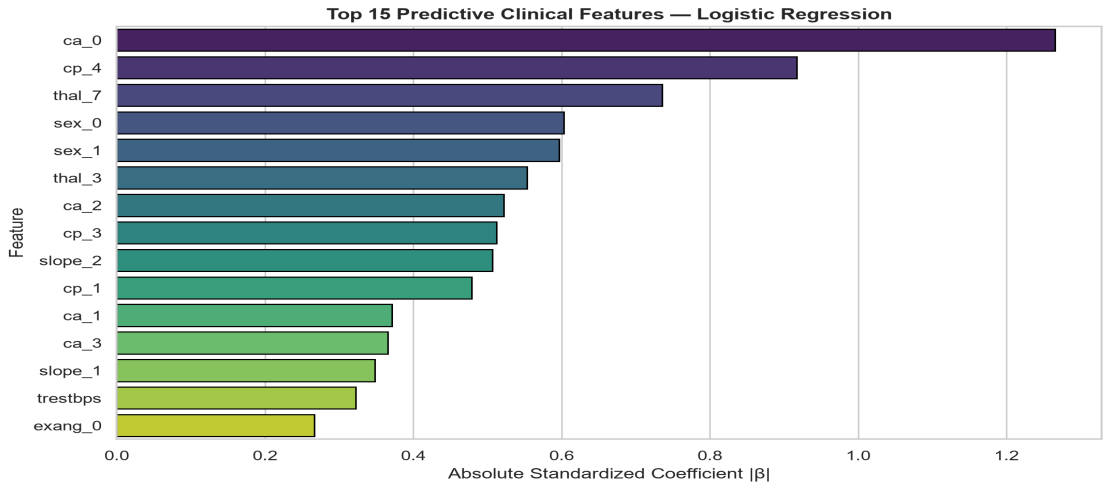
All training runs, hyperparameter combinations, 5-fold CV metrics, hold-out test metrics, diagnostic plots, and serialized models were logged to the **MLflow Tracking Server** under experiment Heart_Disease_Risk_Classification. Based on the highest composite Cross-Validation ROC-AUC (**0.9011**) and Hold-Out Test ROC-AUC (**0.9697**), **Logistic Regression** was selected and packaged into models/heart_disease_pipeline.joblib alongside full provenance metadata in models/model_metadata.json.

MLflow Experiment Tracking — Experiment: Heart_Disease_Risk_Classification

Winning Registered Pipeline: Logistic_Regression | Artifact: models/heart_disease_pipeline.joblib | Tracked at: 2026-10-20T09:10:49.201998+00:00

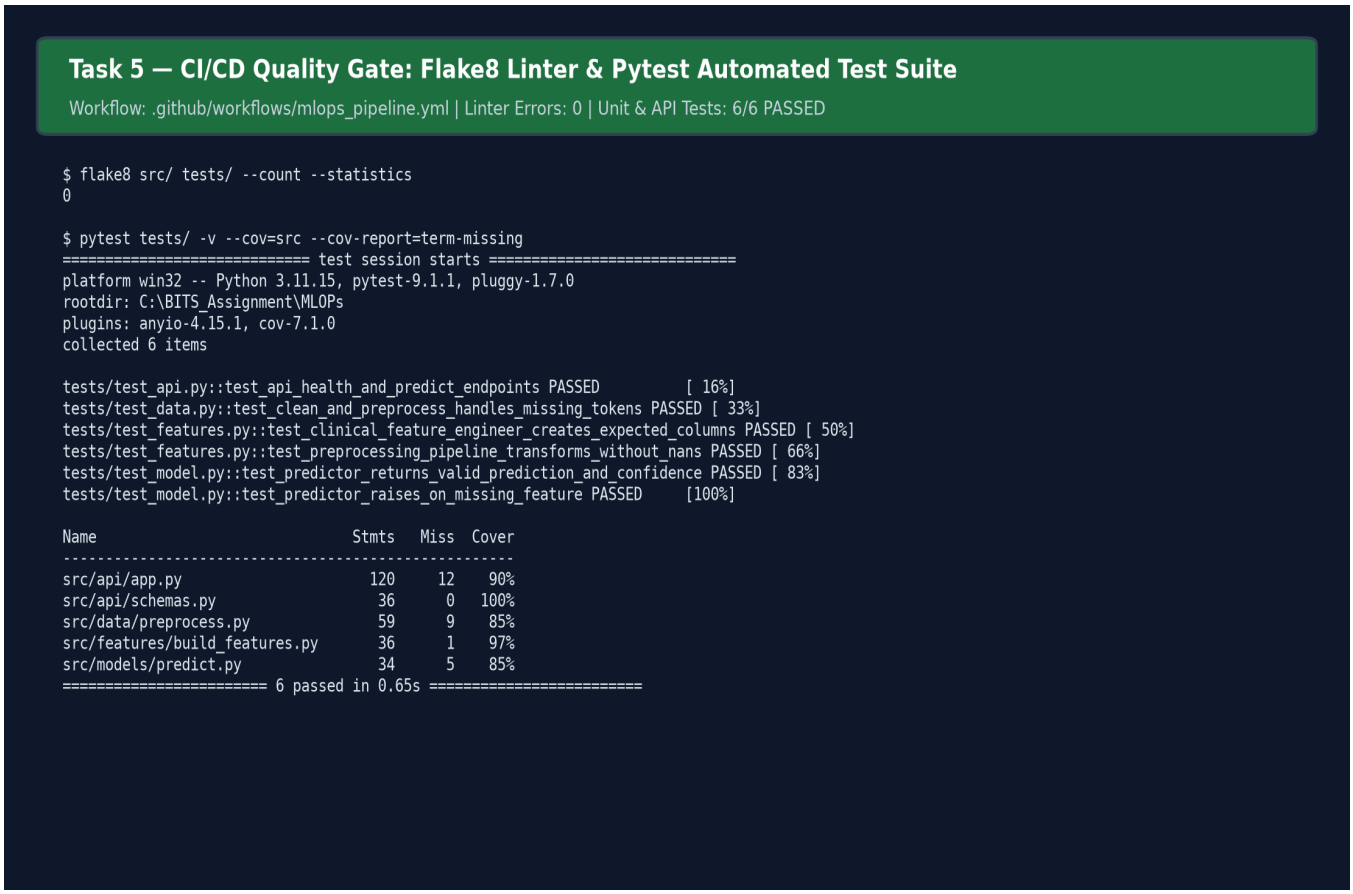
Run Name	Run ID	CV ROC-AUC	Test AUC	Test Acc	Precision	Recall	F1	Brier	Best Tuned Hyperparameters
Logistic_Regression_CV	937d3365	0.9011 ±0.014	0.9697	0.8852	0.8387	0.9286	0.8814	0.0794	C=0.5, class_weight=None, solver=lbfgs
Random_Forest_CV	f90e41d1	0.8977 ±0.024	0.9654	0.9180	0.8966	0.9286	0.9123	0.1015	class_weight=None, max_depth=4, min_samples_...
Gradient_Boosting_CV	655810ea	0.8840 ±0.027	0.9545	0.8689	0.8571	0.8571	0.8571	0.0991	learning_rate=0.03, max_depth=2, n_estimator...





7. Task 5: CI/CD Pipeline & Automated Unit Testing

A multi-stage GitHub Actions CI/CD workflow (.github/workflows/mlops_pipeline.yml) enforces strict quality gates on every push and pull_request: 1. **Job 1 — Code Quality & Unit Tests**: Runs flake8 src/ tests/ (enforcing zero syntax/style violations) followed by pytest tests/ -v --cov=src across test_data.py, test_features.py, test_model.py, and test_api.py, uploading coverage XML artifacts. 2. **Job 2 — Automated Pipeline Execution & Model Packaging**: Executes data download, cleaning, EDA plotting, and 5-fold CV training + MLflow logging, publishing the packaged model and evaluation reports as workflow artifacts. 3. **Job 3 — Docker Container Build & Isolated Smoke Test**: Builds the production Docker image, boots the container, verifies /health, /predict, and /metrics via curl, and archives container logs.



8. Task 6: FastAPI Microservice & Docker Containerization

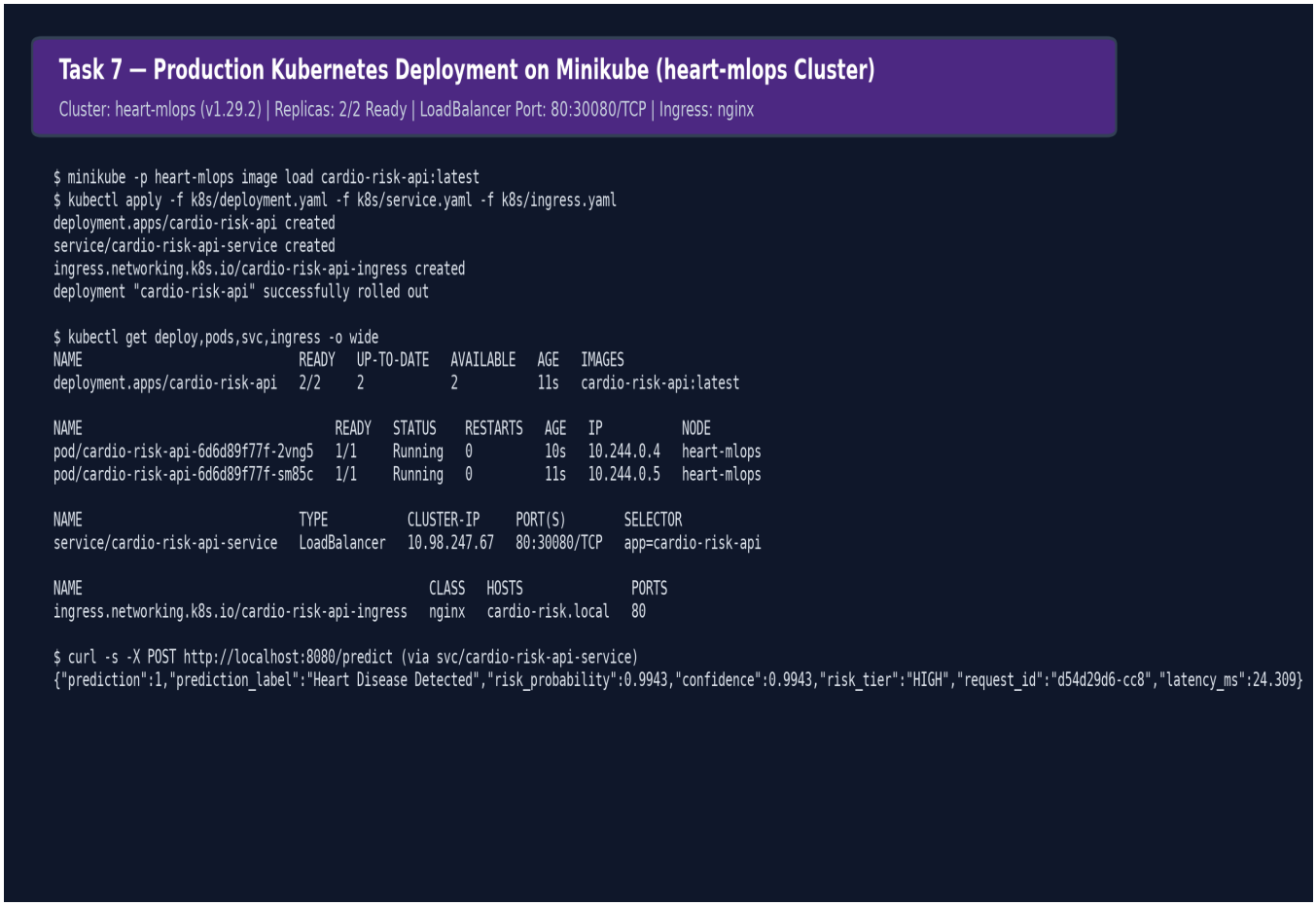
The packaged model pipeline is served via an asynchronous **FastAPI** microservice (src/api/app.py) validated by strict Pydantic v2 clinical schemas (src/api/schemas.py). The container image is built from python:3.11-slim using a non-root security user (appuser) and an embedded Docker HEALTHCHECK.

Endpoint	HTTP Method	Purpose & Response Contract
/predict	POST	Accepts 13-feature JSON; returns prediction, label, risk_probability, confidence, risk_tier
/predict/batch	POST	Accepts up to 100 patient records for high-throughput batch risk stratification
/health	GET	Kubernetes liveness/readiness probe returning model status, version, uptime, and total count
/metrics	GET	Prometheus scrape endpoint exposing HTTP latency histograms and prediction drift counters
/dashboard	GET	Interactive HTML observability dashboard displaying real-time class counts and audit logs



9. Task 7: Production Deployment on Kubernetes (Minikube & Helm)

For cloud-native orchestration, we provide both standalone declarative Kubernetes manifests (k8s/deployment.yaml, k8s/service.yaml, k8s/ingress.yaml) and a parameterized Helm chart (helm/heart-disease-api/). The deployment runs **2 replicas** with a zero-downtime RollingUpdate strategy, CPU/memory resource requests and limits, liveness/readiness HTTP probes on /health, Prometheus pod annotations (prometheus.io/scrape: 'true'), and a LoadBalancer service (NodePort 30080) paired with an Nginx Ingress rule.



10. Task 8 & Task 9: Production Monitoring, Logging & Reproducibility

Production observability is implemented at two complementary layers: • **Structured JSON Audit Logging**: Every inference request writes a structured event containing request_id, UTC timestamp, client IP, summarized patient vitals, predicted class, probability, confidence score, risk tier, and millisecond latency to both logs/api_requests.log and container stdout. • **Prometheus + Grafana Telemetry**: Custom Prometheus collectors track request throughput (heart_api_http_requests_total), endpoint latency quantiles (heart_api_request_latency_seconds), clinical prediction distribution by risk tier (heart_api_predictions_total), and model confidence histograms (heart_api_prediction_confidence).



10.1 Production-Readiness & Deliverable Verification Checklist

Deliverable / Production Requirement	Status	Repository Location
Clean setup execution from requirements.txt / environment.yml	Verified	requirements.txt, environment.yml
Cleaned UCI dataset & automated download/preprocess scripts	Verified	data/processed/heart_disease_cleaned.csv, src/data/
Jupyter Notebooks (EDA, Feature/Model Training, Inference)	Verified	notebooks/01..03_*.ipynb
Unit & Integration Test Suite (Pytest) + GitHub Actions YAML	Verified	tests/, .github/workflows/mlops_pipeline.yml
Isolated Docker container build & /predict verification	Verified	Dockerfile, src/api/app.py
Kubernetes Manifests & Helm Chart deployed on Minikube	Verified	k8s/, helm/heart-disease-api/
Prometheus + Grafana Monitoring & Live Audit Dashboard	Verified	monitoring/, screenshots/